GitHub Action walkthrough

What is GitHub Action?

In a nutshell, it is a workflow automation service provided by GitHub. This service allows automating all kinds of processes and actions related to repositories. Basically, github actions proiveds various automation services related to the repositories(stored code). There are 2 main areas of processes that can be automated. These are:

- **CI/CD (Automating code testing, building, and deployment):** CI (Continuous Integration) is all about integrating new code or updating the changes into an existing code base by building that code, testing the code, and then merging it into the existing code. CD (continuous Delivery/Deployment) is about publishing new versions of apps, packages, or websites automatically after the code has been tested and integrated. It's all about automation and handling the code changes automatically.

- **Code & Repository Management (Automate code reviews, issues, management, etc.):** Basic usage of git and GitHub.

Git

- A free tool you install on your computer to track changes in your code.
- You can save snapshots of your code (called commits), work on different versions at the same time (using branches), and go back to earlier versions if you make a mistake.
- Works with any programming language or project type.

GitHub

- A company that provides cloud storage for your Git repositories (places where your code lives).
- You can store your code online so you don't lose it if your computer crashes, and you can access it from any device.
- It also has tools for teamwork—like reporting bugs, suggesting changes, and managing who can edit code.
- Repositories can be public or private.

GitHub Actions

- An automation service from GitHub that helps you automate tasks related to your code, like testing or deploying it automatically.
- It's part of what makes GitHub useful beyond just storing code.

In short:

Git helps you manage code changes on your computer.
GitHub stores that code online and helps teams work together.
GitHub Actions automates repetitive tasks for your projects.

Git Refresher

Git is a free, open-source version control system (VCS). It is basically used to manage and control the source code. Key features include:

- **Commits:** Saved snapshots of your code at specific points.
- **Branches:** Isolated lines of development for features or fixes (to be covered later).

Git Commits

To be able to work with git, projects need to be attached to Git repositories. Make sure the root folder of the projects is the path where the repos are being initialized for a better development experience.

If we have any specific files or folders that we don't want to include in git then we can specify the files or folders in a **.gitignore** file. Make sure the file is created in the root folder of the git repository.

Commands	Descriptions
git init	Create a git repository for the project. It will create a **.git(invisible)** file, which will hold all git related metadata. Without initialising any repositories, other commands will not work.
git status	Information about the current status of the repository.

git add <file(s)>	Staging changes for the next commit. Defining which changes should be part of the next commit.
git commit	Creates a commit based on the changes that have been staged. It will open a text editor to write the commit message.
git commit -m “COMMIT MESSAGE”	Creating a snapshot(commit) with a short message.
git log	Provides a chronologically ordered list of all the commits has been made.
git checkout <commit-id>	Temporarily move to another(id) commits.
git revert <commit-id>	Revert the changes by creating a new commit. It does not delete any commits.
git reset –hard <commit-id>	Undo changes by deleting all the commits from the current commit to the <commit-id> commit.

Git Branches

Branches are like separate workspaces in a project where we can work on new features or fixes without affecting the main, stable version of your code. We can think of the main branch as the "official" version, and branches as copies where we can try things out safely.

Why use branches?

- Work on something new (like redesigning a website) without breaking what’s already working.
- Let different people work on different tasks at the same time, in their own branches.
- Keep your main branch clean and ready for release.

Commands	Descriptions
git branch <name>	Create a new branch with <name> name with the latest commit targeted by “HEAD”.
git branch	Display the list of all the branches available in the repo. Also shows which branch is currently active(* sign next to the branch).
git checkout <branch-name>	Switch to the <branch-name> branch.

git branch -D <branch-name>	Delete the <branch-name> branch.
git checkout -b <branch-name>	Create a new <branch-name> branch and instantly checkout to that branch.
git merge <name>	Merge all the commits in the <name> branch and append to the main branch. Also, a commit message will be asked for since this command is also similar to commit.

Github Refresher

GitHub stores your Git repositories in the cloud, keeping your code safe, accessible, and easier to share or collaborate on.

What Does GitHub Do?

- **Cloud Repositories:** You can create “storage buckets” in the cloud for your code, called GitHub repositories. It can either be public or private.
- **Backup & Access:** Acts as a safe backup and lets you access your code from different devices easily.
- **Collaboration:** Makes sharing code with others simple and supports teamwork.
- **Extra Features:** Includes tools for project management, bug tracking, and automation (like GitHub Actions).

Commands	Descriptions
git remote add <local-repo-name> <repo-url>	Connect the local repo with the git cloud(GitHub, GitLab, Bitbucket) with the local repo name(default origin) of the GitHub repository, with the <repo-url> GitHub repository link
git remote	A list of all connected remote repositories.
git remote get-url <remote-repo-name>/origin	Get the URL of <remote-repo-name>/origin remote repo.
git remote set-url origin https://username@repo-link	Setting up the account to perform the actions on GitHub.
git push	Upload the local commits to the git cloud.
git pull	Download commits from remote repositories to local repositories.

git push <remote-repo-name>/origin <branch-name>	Upload the local commits of the <branch-name> branch to the git cloud. If the git cloud service doesn't have a <branch-name> branch, then it will create a new branch named <branch-name>.
git push origin <branch-name>	Push the branch to the remote repository (origin).

Github Action Key Elements

- **Workflow:** Automated processes attached to a GitHub repository. Key Points:
  - You can add multiple workflows to a single repository.
  - Workflows are triggered by **events** (e.g., push, pull request, manual trigger).
  - They define what should happen(**events**) and when it should happen.

- **Jobs:** Units of execution inside a workflow. Key Points:
  - A **workflow** contains one or more jobs.
  - Jobs run in **parallel** by **default**, but can be **configured** to run **sequentially**.
  - Each job runs on a runner—a virtual machine (Linux, macOS, Windows) provided by GitHub or self-hosted.
  - Jobs can be conditional (run only if certain conditions are met).

- **Steps:** Individual tasks within a job. Key Points:
  - A **job** contains one or more **steps**.
  - **Steps** run in order, one after another (not in parallel).
  - Each step can be either:
    - A shell command (e.g., npm install).
    - An action (reusable, pre-built script from GitHub Marketplace or custom).
  - Steps can also be conditional.

Example Use Case

- **Event:** A push to the main branch.
- **Workflow:** Runs automated tests.
  - **Job:** Test on Ubuntu
    - **Step 1:** Checkout code.
    - **Step 2:** Install dependencies.
    - **Step 3:** Run test suite.

What is a Workflow?

- A workflow is an automated process you define in our repository.
- It's stored in a .github/workflows/ directory as a YAML file.
- A workflow consists of jobs, which contain steps.

Basic Structure of a Workflow File

```yaml
name: Workflow Name          # Name of the workflow
on: [push]                   # Event that triggers the workflow
jobs:                        # List of jobs
  job-name:                  # Name of the job
    runs-on: ubuntu-latest   # Runner/environment
    steps:                   # List of steps
      - name: Step Name      # Step description
        run: echo "Hello"    # Command to run
```

Triggers (Events)

- Workflows run based on **events** like push, pull_request, or workflow_dispatch (manual trigger).
- We can specify multiple events, e.g., on: [push, workflow_dispatch].

Jobs and Steps

- **Jobs** run on a runner (e.g., Ubuntu, Windows, macOS).
- **Steps** are individual tasks in a job.
- Steps can either:
    - Run a shell command using run
    - Use a prebuilt **Action** with uses

Using Actions

- **Actions** are reusable tasks (e.g., checking out code, setting up Node.js).
- They are used with uses: followed by the action name and version.
- Example: actions/checkout@v3 downloads our repository code.

Running Multiple Jobs

- A workflow can have **multiple jobs**.
- Jobs can run **in parallel** by default.
- We can make jobs run **sequentially** using the needs keyword (e.g., deploy needs test).

Handling Failures

If a step fails, the job stops, and the workflow can be marked as failed.
We can view logs to debug failures directly in the **Actions tab**.

Expressions and Context

- We can access **runtime information** (like repository details) using expressions.
- Syntax: ${{ expression }}
- Example: ${{ github.repository }} gives the repo name.

Key Takeaways:

- Workflows are **YAML files** in .github/workflows/.
- They are triggered by **events** (push, PR, manual, etc.).
- **Jobs** run on runners and contain **steps**.
- **Actions** are reusable components for common tasks.
- Jobs can depend on each other with needs.
- Use **expressions** (${{ }}) to access context data.

Types of Events

We can fine-tune when your workflows run using **activity types** and **filters**, skip them when needed, cancel them if something goes wrong, and manage security for external contributors.

Workflows can be triggered by many different events:

- **Repository events:** Like pushes, pull requests, or issues.
- **General events:** Like scheduled runs or manual triggers (`workflow_dispatch`).

Example:

```yaml
on:
  push:                    # When code is pushed
  pull_request:            # When a pull request is created/updated
  schedule:                # On a schedule
    - cron: '0 2 * * *'    # Daily at 2 AM
  workflow_dispatch:       # Manual trigger in GitHub UI
```

Activity Types

Some events have sub-types to give you more control. For example:

- A `pull_request` can be **opened**, **edited**, **closed**, etc.
- By default, only some types (like `opened`) trigger workflows unless you specify others.

Example: Control which pull request actions trigger the workflow:

```yaml
on:
  pull_request:
    types: [opened, synchronize, reopened]  # Only on open, update, or
reopen
```

Without `types`, it runs only on default activities. Add `closed` if you want it on closure too.

Event Filters

You can restrict when a workflow runs using filters:

- **Branch filters:** Only run for pushes or pull requests to specific branches (e.g., `main`, `dev-*`).
- **Path filters:** Only run if files in certain paths are changed (e.g., ignore changes in `.github/workflows/`).

Example:

Only run when pushing to `main` or `dev-*` branches:

```yaml
on:
  push:
    branches:
      - main
      - 'dev-*'
```

Only run if files in `src/` change, but ignore `docs/`:

```yaml
on:
  push:
    paths:
      - 'src/**'
      - '!docs/**'
```

Skipping Workflows

You can skip a workflow run by adding special keywords to your commit message, like:

- `[skip ci]` or `[skip actions]`
  This is useful for minor changes (like comments) where running tests or deployment isn't needed.

Example commit message:

```
Fixed typo in README [skip ci]
```

This push won't trigger any workflow.

Canceling Workflows

Workflows can be canceled:

- **Automatically:** If a job or step fails (default behavior).
- **Manually:** Via the GitHub UI if you know the workflow will fail or is unnecessary.

Example:

- **Auto-cancel:** If a job fails → whole workflow stops (default).
- **Manual cancel:** Click "Cancel workflow" in GitHub Actions UI if you know it will fail.

Pull Requests from Forks

For security, workflows are **not automatically triggered** for pull requests from forked repositories by first-time contributors. The repository owner must approve the first run manually. After approval, future pull requests from that contributor will run automatically.

Example Combined Workflow:

```yaml
name: CI Pipeline
on:
  push:
    branches: [main, 'release/*']
    paths-ignore: ['docs/**', '**.md']
  pull_request:
    branches: [main]
    types: [opened, synchronize]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Run tests
        run: npm test
```

- Runs on push to `main` or `release/*` branches, but not if only docs/markdown changed.
- Runs on PRs to `main` when opened or updated.
- Skips if commit has `[skip ci]`.
- Can be manually canceled.

```yaml
name: Deploy to Production
on:
  push:
    branches: [main]           # Only main branch
    paths-ignore: ['docs/**']  # Skip if only docs changed
  pull_request:
    branches: [main]           # PRs to main
    types: [opened]            # Only when opened
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Check code
        run: echo "Deploying to production..."
```

- Runs when you push to `main` (unless only docs changed)
- Runs when someone opens a PR to `main`
- Skips if commit has `[skip ci]` in message
- Won't run for first-time contributors from forks until approved
- Can be canceled manually if something goes wrong

Artifacts

Artifacts are files or folders produced by a job that you want to keep or share—like a built website, app binary, or log files.

Example:

- A `build` job generates a `dist` folder with website files.
- You upload artifacts using the `upload-artifact` action.
- You can download them manually from GitHub or use them in another job with the `download-artifact` action.

Job Outputs

Outputs are simple values (like text or numbers) that one job produces and another job can use—for example, a generated filename, version number, or status.

Example:

- In a `build` job, you can output the name of a generated JavaScript file.
- Another job (like `deploy`) can access that filename using the `needs` context.

Caching Dependencies

Caching speeds up workflows by reusing downloaded dependencies (like `node_modules`) across jobs and workflow runs, so you don't download them every time.

Example:

- Use the `cache` action to store the npm cache.
- The cache key often uses a hash of `package-lock.json`—so the cache updates only when dependencies change.

```yaml
# Workflow name (shows in GitHub Actions tab)
name: Build, Test, and Deploy
# Trigger: Run on push to main branch
on:
  push:
    branches: [ main ]
# Jobs run in parallel by default, but can have dependencies
jobs:
  # Job 1: Run tests
  test:
    # Use GitHub's Ubuntu runner
    runs-on: ubuntu-latest
    steps:
      # Step 1: Get code from repository
      - name: Checkout code
        uses: actions/checkout@v4
      # Step 2: Cache npm dependencies for faster runs
      - name: Cache dependencies
        uses: actions/cache@v4
        with:
          # npm cache location on Linux
          path: ~/.npm
          # Cache key changes when package-lock.json changes
          key: npm-${{ hashFiles('package-lock.json') }}
      # Step 3: Install dependencies (uses cache if available)
      - name: Install dependencies
        run: npm ci  # 'ci' for clean install, faster than 'install'
      # Step 4: Run test scripts
      - name: Run tests
        run: npm test
```

```yaml
  # Job 2: Build the project (depends on test job)
  build:
    runs-on: ubuntu-latest
    needs: test  # Wait for test job to complete

    # Define outputs that other jobs can access
    outputs:
      script-file: ${{ steps.publish.outputs.script-file }}

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      # Cache dependencies (same as test job, reuses same cache)
      - name: Cache dependencies
        uses: actions/cache@v4
        with:
          path: ~/.npm
          key: npm-${{ hashFiles('package-lock.json') }}

      - name: Install dependencies
        run: npm ci

      # Build step (creates dist/ folder with website files)
      - name: Build project
        run: npm run build

      # Create a job output (simple value, not a file)
      - name: Publish JS filename as output
        id: publish  # Give step an ID to reference later
        run: |
          # Find first JavaScript file in dist/ folder
          FILENAME=$(find dist -name "*.js" | head -1)
          # Set step output (accessible via
steps.publish.outputs.script-file)
          echo "script-file=$FILENAME" >> $GITHUB_OUTPUT

      # Upload build files as artifact for download/use in other jobs
      - name: Upload build artifacts
        uses: actions/upload-artifact@v4
        with:
          name: dist-files  # Name to identify artifact
          path: dist/       # Folder to upload
```

```yaml
  # Job 3: Deploy (depends on build job)
  deploy:
    runs-on: ubuntu-latest
    needs: build  # Wait for build job to complete

    steps:
      # Download artifact from build job
      - name: Download build artifacts
        uses: actions/download-artifact@v4
        with:
          name: dist-files  # Must match upload artifact name

      # Access output from build job using 'needs' context
      - name: Show output from build job
        run: echo "JS file from build job: ${{
needs.build.outputs.script-file }}"

      # List files to confirm artifact was downloaded
      - name: List downloaded files
        run: ls -la

      # Placeholder for actual deployment
      - name: Deploy (example step)
        run: echo "Deploying files to web server..."
        # Real deployment would use specific provider actions here
        # Example: AWS S3, Netlify, Vercel, etc.
```

What this workflow does:

Test job: Checks code, caches dependencies, runs tests.

Build job:

- Builds the project.
- Outputs a generated JS filename.
- Uploads the `dist/` folder as an artifact.

Deploy job:

- Downloads the artifact.
- Uses the output from the build job.
- Simulates deployment.

Real-World Extensions

Replace the `Deploy` step with actual deployment actions:

```
- name: Deploy to Netlify
  uses: nwtgck/actions-netlify@v2
  with:
    publish-dir: './dist'
    production-branch: main
```

This workflow demonstrates the complete CI/CD pipeline using all three key data concepts: artifacts, outputs, and caching.

GitHub Actions: Environment Variables & Secrets

Environment Variables

Dynamic values are injected into the code/workflows that can change between environments (testing vs staging vs production).

Common uses:

- Database connection strings
- API endpoints
- Port numbers
- Configuration settings

Where to define them in workflows:

- **Workflow level:** Available to all jobs
- **Job level:** Available only to that job
- **Step level:** Available only to that step

Example:

```yaml
# Workflow-level environment variable
env:
  DB_NAME: myapp-database

jobs:
  test:
    # Job-level environment variables
    env:
      PORT: 3000
      DB_HOST: test-cluster.mongodb.net

    steps:
      # Access in workflow file
      - name: Show DB info
        run: echo "Connecting to $DB_NAME on port $PORT"

      # Code automatically accesses process.env.PORT, process.env.DB_HOST
```

Secrets

Sensitive environment variables (passwords, API keys, tokens) are stored securely in GitHub.

Where to store them:

- **Repository secrets:** Available to all workflows in that repo
- **Environment secrets:** Available only to jobs using that environment
- **Organization secrets:** Available to all repos in the org

Example:

```yaml
jobs:
  deploy:
    env:
      # Reference secrets - GitHub hides them in logs
      DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
      API_KEY: ${{ secrets.API_KEY }}

    steps:
      - name: Deploy with secret
        run: ./deploy.sh --key "$API_KEY"
```

Environments

Named environments (testing, staging, production) with their own secrets and rules.

Features:

- Environment-specific secrets
- Branch protection rules
- Manual approvals
- Wait timers

Example:

```
jobs:
  test:
    # Use testing environment with its own secrets
    environment: testing

  deploy-prod:
    # Use production environment with approval required
    environment: production
    needs: test
```

Combined Workflow Example

```
name: Test and Deploy with Environments
on:
  push:
    branches: [ main, dev ]
  pull_request:
    branches: [ main ]
# Workflow-level environment variables (available to all jobs)
env:
  APP_NAME: "my-api-app"
  NODE_ENV: "production"

jobs:
  test:
    runs-on: ubuntu-latest
    # Use the testing environment with its secrets
    environment: testing
```

```yaml
  # Job-level environment variables
  env:
    PORT: 3000
    DB_NAME: "test-db"

  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '18'
    - name: Install dependencies
      run: npm ci
    - name: Run tests with environment variables
      env:
        # Secrets from testing environment
        DB_USER: ${{ secrets.DB_USER }}
        DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
        DB_HOST: ${{ secrets.DB_HOST }}
      run: |
        echo "Testing $APP_NAME on port $PORT"
        echo "DB connection: $DB_USER@$DB_HOST"
        npm test
    - name: Upload test results
      uses: actions/upload-artifact@v4
      with:
        name: test-results
        path: test-results/

deploy-staging:
  runs-on: ubuntu-latest
  environment: staging
  needs: test
  # Only run from the main branch
  if: github.ref == 'refs/heads/main'

  env:
    DEPLOY_URL: "https://staging.myapp.com"

  steps:
    - name: Checkout code
      uses: actions/checkout@v4
```

```
      - name: Deploy to staging
        env:
          # Staging environment secrets
          STAGING_API_KEY: ${{ secrets.STAGING_API_KEY }}
          STAGING_DB_URL: ${{ secrets.STAGING_DB_URL }}
        run: |
          echo "Deploying to $DEPLOY_URL"
          ./deploy-staging.sh "$STAGING_API_KEY"

  deploy-production:
    runs-on: ubuntu-latest
    environment: production
    needs: deploy-staging
    # Production requires manual approval (set in GitHub environment
settings)

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Deploy to production
        env:
          # Production environment secrets
          PROD_API_KEY: ${{ secrets.PROD_API_KEY }}
          PROD_DB_URL: ${{ secrets.PROD_DB_URL }}
        run: |
          echo "🚀 Deploying to production!"
          ./deploy-production.sh "$PROD_API_KEY"
```

Key Syntax Reference

Accessing values in workflow file:

```
# Environment variables
run: echo "Port is $PORT"                    # Shell interpolation
run: echo "Port is ${{ env.PORT }}"          # GitHub context

# Secrets
run: echo "Using API key: ${{ secrets.API_KEY }}"
```

Accessing in code (Node.js example):

```
// Automatically available to your application
const port = process.env.PORT;
const dbHost = process.env.DB_HOST;
const dbPassword = process.env.DB_PASSWORD;  // From secrets
```

Best Practices

- Never commit secrets to version control
- Use environment-specific secrets (testing vs production)
- Set branch protection rules for production environments
- Use the `secrets` context instead of hardcoding sensitive values
- Review the environment settings in GitHub before using them

This setup ensures secure, flexible workflows that adapt to different environments while keeping sensitive data protected.

Conditional Execution with `if`

We can add `if` conditions to jobs or steps to control when they run.

Example: Step runs only if a previous step failed

```
steps:
  - name: Run tests
    id: run-tests
    run: npm test

  - name: Upload test report
    if: steps.run-tests.outcome == 'failure'
    run: echo "Uploading report..."
```

Continue on Error

Allows a job to continue even if a step fails.

Example: Don’t stop the job if this step fails

```yaml
steps:
  - name: Run risky script
    continue-on-error: true
    run: ./risky-script.sh
```

Matrix Jobs

Run the same job with different configurations (e.g., OS, Node versions).

Example: Test on multiple Node versions and OS

```yaml
jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        node-version: [14, 16, 18]
    steps:
      - name: Use Node ${{ matrix.node-version }}
        uses: actions/setup-node@v3
        with:
          node-version: ${{ matrix.node-version }}
```

Reusable Workflows

Define a workflow once and reuse it in others.

Reusable workflow (`.github/workflows/deploy.yml`):

```yaml
on:
  workflow_call:
    inputs:
      artifact-name:
        required: true
        type: string

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Download artifact
        uses: actions/download-artifact@v4
        with:
          name: ${{ inputs.artifact-name }}
```

Calling the reusable workflow:

```yaml
jobs:
  deploy:
    uses: ./.github/workflows/deploy.yml
    with:
      artifact-name: 'dist-files'
```

Combined Workflow Example

```yaml
name: Combined Example
on: [push]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Lint
        run: npm run lint
```

```yaml
  test:
    runs-on: ${{ matrix.os }}
    needs: lint
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        node-version: [16, 18]
    steps:
      - name: Setup Node
        uses: actions/setup-node@v3
        with:
          node-version: ${{ matrix.node-version }}

      - name: Run tests
        id: run-tests
        run: npm test

      # Only upload report if tests fail
      - name: Upload test report
        if: steps.run-tests.outcome == 'failure'
        run: echo "Test failed -- uploading report..."

      # This step won't stop the job if it fails
      - name: Optional step
        continue-on-error: true
        run: ./optional-script.sh

  deploy:
    runs-on: ubuntu-latest
    needs: test
    if: github.ref == 'refs/heads/main'  # Only deploy from main
    steps:
      - name: Deploy to production
        run: echo "Deploying..."
```

Key Takeaways

- Use `if` to run steps/jobs conditionally.
- Use `continue-on-error` to prevent step failure from stopping the job.
- Use `matrix` to test across multiple environments.
- Use **reusable workflows** to avoid repeating code.

Using Containers with GitHub Actions

What Are Containers?

Containers package your code together with its execution environment (software, dependencies, settings). This ensures your application runs the same way everywhere.

- **Example:** A Node.js app bundled with Node.js runtime and Linux OS in one package. (`Docker` file)

Running Jobs in Containers

Instead of running jobs directly on GitHub's runner machines, you can run them in custom container environments you control.

- **Why use it:** Full control over installed software, versions, and environment setup.
- **Example:** Using a pre-made `node:16` image instead of installing Node.js manually.

Service Containers

Extra containers that run alongside your job to provide services like databases, only available while your workflow runs.

- **Example:** A MongoDB database container for testing, shut down after tests complete.
- **Benefit:** No need for external testing servers; everything runs locally in isolation.

Key Configuration Differences

- **Job in container + Service container:** Use service label as hostname (e.g., `mongodb`)
- **Job on runner + Service container:** Use `localhost:port` and map ports explicitly
-

Combined Workflow Example:

```yaml
name: Test with Containers
on:
  push:
    branches: [ main ]
jobs:
  test:
    # Run this job inside a container
    container:
      image: node:16  # Uses Node.js 16 environment
```

```
    # Service container that runs alongside the job
    services:
      mongodb:  # Label for the service
        image: mongo  # MongoDB image from Docker Hub
        env:
          # Set credentials for the test database
          MONGO_INITDB_ROOT_USERNAME: root
          MONGO_INITDB_ROOT_PASSWORD: example
        # Port mapping only needed if job runs on runner (not in container)
        # ports:
        #   - 27017:27017
    env:
      # Connection string for MongoDB service
      # When job runs in container, use service label as hostname
      MONGODB_URL: mongodb://root:example@mongodb:27017/mydb
      # If job runs on runner (not in container), use:
      # MONGODB_URL: mongodb://root:example@localhost:27017/mydb
    steps:
      # Checkout code (works in containers too)
      - name: Checkout repository
        uses: actions/checkout@v3
      # Install dependencies
      - name: Install dependencies
        run: npm ci
      # Run tests (will connect to MongoDB service container)
      - name: Run tests
        run: npm test
      # Example: Run integration tests with service
      - name: Run integration tests
        run: |
          # The app will connect to MongoDB at the URL above
     npm run test:integration
```

Key Points to Remember:

- **Containers give environment control** - Define exactly what software is installed
- **Service containers are temporary helpers** - Perfect for test databases
- **Networking differs** based on whether your job runs in a container or on the runner
- **Public images from Docker Hub** are readily available (node, mongo, postgres, etc.)
- **You don't need containers** if the default runner environment has everything you need

Custom Actions

Why Build Custom Actions?

- **Simplify Workflows:** Group repeated steps into one action.
- **Reuse Logic:** Use the same action across multiple jobs or workflows.
- **No Public Action Available:** When existing actions don't meet your needs.
- **Full Control:** Build exactly what you need with your own logic.

Types of Custom Actions:

Composite Actions

- **Purpose:** Combine multiple workflow steps into one reusable action.
- **Use Case:** Repeated steps like caching and installing dependencies.

Example Structure:

```yaml
# .github/actions/cache-deps/action.yml
name: 'Cache and Install Dependencies'
description: 'Caches and installs npm dependencies'
inputs:
  caching:
    description: 'Whether to use caching'
    required: false
    default: 'true'
outputs:
  used-cache:
    description: 'Whether cache was used'
runs:
  using: 'composite'
  steps:
    - name: Cache dependencies
      if: ${{ inputs.caching == 'true' }}
      uses: actions/cache@v3
      with:
        path: node_modules
        key: npm-${{ runner.os }}-${{ hashFiles('package-lock.json') }}
    - name: Install dependencies
      shell: bash
      run: |
        npm ci
        # Set output (alternative to composite action steps)
        echo "::set-output name=cache::${{ inputs.caching }}"
      id: install
```

JavaScript Actions:

- **Purpose:** Write action logic in JavaScript (Node.js).
- **Use Case:** More complex tasks, like deploying files to AWS S3.

Example Structure:

```yaml
# .github/actions/deploy-s3/action.yml
name: 'Deploy to AWS S3'
description: 'Uploads static files to an S3 bucket'
inputs:
  bucket:
    description: 'S3 bucket name'
    required: true
  folder:
    description: 'Local folder to upload'
    required: true
outputs:
  website-url:
    description: 'URL of deployed website'
runs:
  using: 'node16'
  main: 'index.js'
```

```javascript
// index.js
const core = require('@actions/core');
const exec = require('@actions/exec');

async function run() {
  try {
    // Get inputs
    const bucket = core.getInput('bucket');
    const folder = core.getInput('folder');

    // Use AWS CLI to sync folder to S3
    // AWS credentials should be set as environment variables in workflow
    await exec.exec(`aws s3 sync ${folder} s3://${bucket} --region
us-east-1`);

    // Set output
    const websiteUrl =
`http://${bucket}.s3-website-us-east-1.amazonaws.com`;
```

```
    core.setOutput('website-url', websiteUrl);
    core.notice(`Website deployed to: ${websiteUrl}`);

  } catch (error) {
    core.setFailed(`Deployment failed: ${error.message}`);
  }
}

run();
```

Docker Actions

- **Purpose:** Package any language/tool in a container.
- **Use Case:** When you need a specific environment (e.g., Python with AWS SDK).

Example Structure:

```yaml
# .github/actions/deploy-s3-docker/action.yml
name: 'Deploy to S3 (Docker)'
description: 'Uploads files to S3 using a Python container'
inputs:
  bucket:
    description: 'S3 bucket name'
    required: true
outputs:
  website-url:
    description: 'URL of deployed website'
runs:
  using: 'docker'
  image: 'Dockerfile'
```

```
# Dockerfile
FROM python:3.9
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY deploy.py .
ENTRYPOINT ["python", "/deploy.py"]
```

```python
# deploy.py
import os
import boto3
from botocore.exceptions import ClientError

# Get inputs from environment variables (GitHub Actions automatically sets these)
bucket = os.environ.get('INPUT_BUCKET')
folder = os.environ.get('INPUT_FOLDER', './dist')

# Set output using GitHub Actions command syntax
website_url = f"http://{bucket}.s3-website-us-east-1.amazonaws.com"
print(f"::set-output name=website-url::{website_url}")
print(f"Deploying {folder} to {bucket}")
```

Combined Workflow Example

```yaml
# .github/workflows/ci-cd.yml
name: CI/CD with Custom Actions
on:
  push:
    branches: [main]
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      # 1. Checkout code
      - name: Checkout
        uses: actions/checkout@v4
      # 2. Use custom COMPOSITE action to cache & install deps
      - name: Get Dependencies
        uses: ./.github/actions/cache-deps
        id: deps
        with:
          caching: 'true'
      # 3. Build the project
      - name: Build
        run: npm run build
      # 4. Upload build artifacts
      - name: Upload Artifacts
        uses: actions/upload-artifact@v4
        with:
          name: dist-files
          path: dist/
  deploy:
```

```yaml
    runs-on: ubuntu-latest
    needs: build-and-deploy
    steps:
      # 1. Download build artifacts
      - name: Download Artifacts
        uses: actions/download-artifact@v4
        with:
          name: dist-files
          path: dist/

      # 2. Use a custom JAVASCRIPT action to deploy to S3 (Staging)
      - name: Deploy to Staging (JS Action)
        uses: ./.github/actions/deploy-s3-js
        id: deploy-js
        with:
          bucket: 'my-staging-bucket'
          folder: './dist'
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}

      # 3. Display deployed URL
      - name: Show Staging URL
        run: echo "Staging website: ${{ steps.deploy-js.outputs.website-url
}}"

      # 4. Use a custom DOCKER action to deploy to S3 (Production)
      - name: Deploy to Production (Docker Action)
        uses: ./.github/actions/deploy-s3-docker
        id: deploy-docker
        with:
          bucket: 'my-production-bucket'
          folder: './dist'
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}

      # 5. Display deployed URL
      - name: Show Production URL
        run: echo "Production website: ${{
steps.deploy-docker.outputs.website-url }}"
```

Key Takeaways

- **Composite Actions:** Best for grouping steps, no coding needed.
- **JavaScript Actions:** Use when you're comfortable with JS/Node.js.
- **Docker Actions:** Use any language or tool inside a container.
- **All actions** can accept inputs and produce outputs.
- **Actions can be local** (in our repo) or public (published to the marketplace).

Important: Always verify the imports are actually used in the code to avoid errors and keep actions clean.

GitHub Actions Security Essentials

Script Injection

- **Problem:** User-generated content (like issue titles) can inject malicious code into workflow shell commands
- **Example:** If a workflow uses `${{ github.event.issue.title }}` directly in a run command, someone could create an issue with title: `A"; echo "got your secrets`
- **Solution:** Store user input in environment variables first before using them in commands

Malicious Third-Party Actions

- **Problem:** Actions from untrusted sources can steal secrets or damage your repository
- **Risk Levels:**
    - Highest security: Use only your own/organization's actions
    - Medium security: Use actions from verified creators (blue checkmark)
    - Lowest security: Use any public action (inspect code first)
- **Best Practice:** Always review action code from unknown authors

Permissions Management

- **Problem:** Workflows have overly permissive access by default
- **Key Concepts:**
    - GitHub automatically provides a `GITHUB_TOKEN` for API access
    - You can restrict what this token can do using `permissions` settings
    - Default is "read-write" for most areas
- **Solution:** Set minimal required permissions at job or workflow level

OpenID Connect for Third-Party Services

- **Problem:** Hard-coded secrets for services like AWS can be stolen or grant excessive permissions
- **Solution:** Use OpenID Connect to request short-lived, scoped permissions dynamically
- **Example:** Instead of storing AWS keys, configure GitHub as an identity provider in AWS and request temporary credentials

Combined Workflow Example

```yaml
name: Secure Workflow Example
on:
  issues:
    types: [opened]

# 1. SET WORKFLOW-LEVEL PERMISSIONS (Principle of Least Privilege)
permissions:
  contents: read      # Only need to read code
  issues: write       # Only need to write to issues
  id-token: write     # Required for OpenID Connect

jobs:
  process-issue-and-deploy:
    runs-on: ubuntu-latest

    # 2. SET JOB-SPECIFIC PERMISSIONS (can override workflow-level)
    permissions:
      issues: write
      id-token: write
      contents: read

    steps:
      # 3. SECURELY HANDLE USER INPUT (Prevent Script Injection)
      - name: Extract issue data securely
        env:
          # Store user input in environment variable FIRST
          ISSUE_TITLE: ${{ github.event.issue.title }}
          ISSUE_BODY: ${{ github.event.issue.body }}
        run: |
          # Now use the environment variables - much safer!
          echo "Processing issue: $ISSUE_TITLE"
```

```
          # Safe comparison - no direct user input in shell logic
          if [[ "$ISSUE_TITLE" == *bug* ]]; then
            echo "This is a bug report"
            echo "bug" >> labels.txt
          fi

      # 4. USE TRUSTED ACTIONS WITH PINNED VERSIONS
      - name: Checkout code (using verified GitHub action)
        uses: actions/checkout@v4  # From verified creator
        # Always pin to specific version, not @main or @latest

      # 5. OPENID CONNECT FOR AWS (No hard-coded secrets)
      - name: Configure AWS credentials
        if: contains(github.event.issue.title, 'deploy')
        uses: aws-actions/configure-aws-credentials@v2  # AWS official
action
        with:
          role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsRole
          aws-region: us-east-1
          # No access keys needed!

      # 6. SAFE DEPLOYMENT WITH MINIMAL PERMISSIONS
      - name: Deploy to S3 (if deployment issue)
        if: contains(github.event.issue.title, 'deploy')
        run: |
          # AWS credentials are now available via OpenID Connect
          # Temporary and scoped only to what the role allows
          aws s3 sync ./ s3://my-bucket/

      # 7. SAFELY INTERACT WITH GITHUB API
      - name: Add label to issue
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}  # Auto-generated,
scoped by our permissions
        run: |
          # The token only has 'issues: write' permission due to our
settings
          curl -X POST \
            -H "Authorization: token $GITHUB_TOKEN" \
            -H "Accept: application/vnd.github.v3+json" \
            https://api.github.com/repos/${{ github.repository
}}/issues/${{ github.event.issue.number }}/labels \
            -d '{"labels":["processed"]}'
```

```
      # 8. EXAMPLE OF AVOIDING UNSAFE PATTERNS (COMMENTED OUT)
      # - name: UNSAFE - DON'T DO THIS
      #   run: |
      #     # NEVER do this - direct user input in command
      #     echo "${{ github.event.issue.title }}" | grep "bug"
      #
      #     # This allows injection like: title: '"; rm -rf /; #'

# 9. REPOSITORY SETTINGS TO CONSIDER (Outside workflow file):
# - Set default workflow permissions to "read repository contents"
# - Require approval for first-time contributors
# - Limit which actions can be used (organization/verified only)
```

Key Takeaways from the Example:

- **Environment Variables First:** Always store user input in env vars before use
- **Minimal Permissions:** Grant only what's needed (issues: write, not issues: write + contents: write)
- **Trusted Actions:** Use verified creators and pin versions
- **No Hard-Coded Secrets:** Use OpenID Connect for cloud services
- Safe API Interaction: GitHub token permissions match your declared permissions